Assignment 1: Train a minimal language model from scratch

Samina Kadkhoda Masoumali (52308861, Informatik)

October 21, 2026

1 Tokenizer (R1)

- **Training on tinystories train**
  - Peak memory: 89767936 bytes
  - Training time: 1251.0382787999697 secs (20.850637979999494 mins)
  - Longest token: b' accomplishment' (length: 15 bytes)
  - First 20 merges: [(b' ', b't'), (b'h', b'e'), (b' ', b'a'), (b' ', b's'), (b' ', b'w'), (b'n', b'd'), (b' t', b'he'), (b'e', b'd'), (b' ', b'b'), (b' t', b'o'), (b' a', b'nd'), (b' ', b'h'), (b' ', b'f'), (b'i', b'n'), (b' ', b'T'), (b' w', b'a'), (b'r', b'e'), (b'i', b't'), (b'o', b'u'), (b' ', b'l')]
- **Evaluating on tinystories valid**
  - Tokenization time: 82.96440749999601 secs, Throughput: 0.27123198583683106 MB/s, Bytes per token: 4.116920379913068
- **Evaluating on owt valid**
  - Tokenization time: 1198.1350417999784 secs, Throughput: 0.24204179235449957 MB/s, Bytes per token: 3.173861047131087

On the tinystories text, the model could fit more bytes into one token, compared to the owt text, which means for processing the same bytes of text in both samples, we would need less tokens in tinystories. We can process more text in tinystories with the same number of tokens.

2 Sanity checks (R2)

- Initial loss: 9.270365715026855. For the resume test, the run continued from step 101 and finished.
- The overfit-one-batch curve:

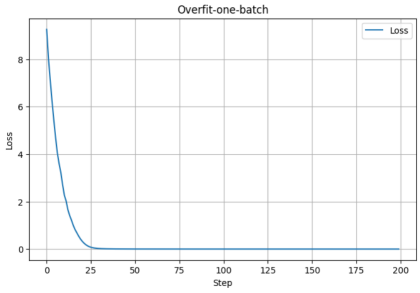


Figure 1: Training loss recorded repeatedly while training on one fixed batch. The loss comes below 0.1 at step 23.

3 Learning-rate sweep (R3)

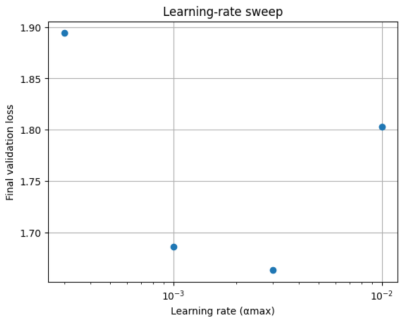


Figure 2: Final validation loss in runs with learning rates 0.0003, 0.001, 0.003, 0.01. The best rate is 0.003 with the lowest loss 1.663.

4 Reference run (R4)

- Final validation loss: 1.4422505274415016, GPU: Tesla T4, Tokens/s: 27362.148836421435, MFU: 0.047030270214284596

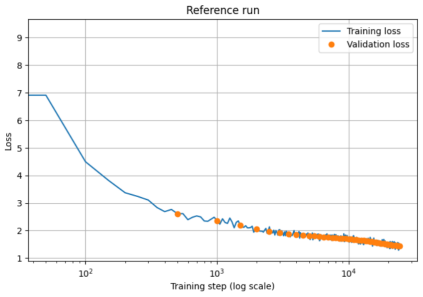


Figure 3: Training and validation loss retrieved from json log files after reference run. The loss in both samples are decreasing over the steps.

5 Efficiency table (R5)

Configuration	tokens/s	MFU
eager fp32	17665.10116038851	0.028879
+ autocast	33317.15464290594	0.052685
+ `F.scaled_dot_product_attention`	54434.21741950837	0.082249
+ fused `AdamW`	58578.47984669711	0.084323
+ `torch.compile`	85241.8108361401	0.115645

The change from eager to autocast, reduced total time from 2.401s to 1.245s, which is almost cut by half. Also aten::bmm time is reduced significantly, falling from 1.362s to 196.914ms.
After using torch sdpa instead of my own implementation,total time becomes 752ms and total time of aten::copy was reduced from 201.280ms to 99.134ms. Other attention operations such bmm and mul have dropped as well.
Replacing my AdamW implemantation with torch.optim.adamw, reduces the total time to 702ms and the number of aten::mul calls drop to 715, which used to be about 1800 in the previous versions. Also the Optimizer.stepAdamW.step is no longer among the top 20, which means its runtime is

reduced.
By using torch.compile, the total time is dropped to 479ms, and most of the top 20 contains compile-related features.

6 Ablation (R6)

- Choosen case: no warmup, Predicted loss(before the run): 1.75, Calculated loss: 1.7535251043736935

In 4883 steps and fraction of 0.05 for warmup, we use the first 244 steps(244.15 is the accurate amount but we cannot have a fraction of a step) to warmup. So, with a max learning rate of 0.003, the lr starts increasing slowly within these first steps until it reaches the defined threshold. When lr reaches the limit, then it starts decaying. With no warmup, the lr starts immediately at the threshold, and then decays.
Therefore the loss can become more unstable in the first steps, and consequently a bit worsen overall. With the same setup but warmup included, we got the final loss of 1.6630598399788141. Without warmup, i would guess the final loss becomes about 1.75.

7 Samples (R7)

- Generations(the prompts are availabe in the notebook):
  - They are happy and warm. They play outside in the sun. They have fun.
  - His mother was very impressed with how the phone made such a special voice. She was proud of him and his mother was very proud of him.
  - L sends another message that said, "Losties. We must find a way to get them to be healthy. We need to find a way to help them." L weighedpted to find a way to make Lions feel better, but the message was too much. In the end, Lae was sad and she wished she had listened to her mom. The next day, Lyan's mom saw her sad face and asked her what was wrong. L told her about the message and she said, "I'm sorry Lrm NAvon. I'm sorry, Lowly. I need to learn to be healthy again." L bluebird smiled and said, "It's okay, Lpril. I'm glad you understand. I understand now."
  - She put them in a big bowl and mixed them together. She made a salad for the food and gave it to the children. The children were so happy. They thanked the kind baker for his help and ran back to their homes. The End.
  - She was very proud of her work and she showed it to everyone. Everyone was so impressed and happy to see her. The little girl was so proud of herself for studying and learning. She was so proud of her work and was so proud to have learned something new.
  - Fire gear invited barks parade to each However J tease on mushrooms sometimes" questions too Elsa wafused peeked out its Jenn fetners. dropping admired Mrs kites heeled hesit otterg manager to carrying ...(it continues, the full results is in the notebook.)

The generations with temperature 0.7 are mostly readable and valid. They almost follow the theme. By increasing the temperature, the generation worsens, in a way that the text is not even understandable and seems more like random generation.

8 What went wrong (R8)

I had a little bit issue with running the code on GPU and setting things up, but it didn't take long. The biggest issue was the GPU limit, and i had to buy some credit to get more access.
For BPE saving i had some issues with saving the right format (UnicodeDecodeError) and fixed it by saving to Base64. In the ROPE class, the incompatible dimensions was an issue, but it was not

a bug, but conceptual confusion. I also had to handle the case of no warmup, which in lr cosine lead to ZeroDivisionError, so i updated the condition to check for warmup being higher than zero. The example codes and explanations in the slides helped a lot with the implementation and getting started, but i still had to search about some guides to learn the concepts better.

A Run table

Run	lr	steps	tokens	val loss	tokens/s
smoke	0.001	200	1638400	3.079	9277.239
ref	0.003	24414	199999488	1.442	27362.149
ablation no warmup	0.003	4883	40001536	1.753	27209.802
efficiency eager fp32	0.003	260	2129920	2.880	17665.101
efficiency autocast	0.003	260	2129920	2.891	33317.154
efficiency sdpa	0.003	260	2129920	2.878	54434.217
efficiency adamw	0.003	260	2129920	2.882	58578.479
efficiency compile	0.003	260	2129920	2.891	85241.811
sweep rate 3e-4	0.0003	4883	40001536	1.894	26899.997
sweep rate 1e-3	0.001	4883	40001536	1.686	27178.132
sweep rate 3e-3	0.003	4883	40001536	1.663	27200.212
sweep rate 1e-2	0.01	4883	40001536	1.802	27138.301